# 상속 길잡이 카카오 챗봇 구축 가이드

Oct 13, 2026 · @sun

## 결론: 로컬 터널 방식은 컴퓨터를 켜 둘 필요가 있고, Render 배포(6단계) 후에는 필요 없습니다

**네, 현재 구성에서는 두 가지가 모두 켜져 있어야 챗봇이 응답합니다.** 카카오 서버는 챗봇 사용자가 말을 걸 때마다 스킬 주소( `https://….trycloudflare.com/skill` )로 요청을 보내고, 그 요청이 터널을 거쳐 내 컴퓨터의 서버( `127.0.0.1:8001` )에 도착해야 응답이 만들어지기 때문입니다.

| 구성 요소 | 역할 | 꺼지면 |
|---|---|---|
| uvicorn (127.0.0.1:8001) | main.py 스킬 서버 | 터널은 살아 있어도 응답 없음 → 카카오 오류 |
| cloudflared 터널 | 카카오 → 내 컴퓨터 통로 | 카카오가 서버에 도달 못 함 → 오류 |
| 컴퓨터 전원·인터넷 | 위 둘의 실행 환경 | 절전·종료·네트워크 끊김 시 둘 다 중단 |

- 터널 주소( `trycloudflare.com` )는 **재시작할 때마다 바뀌고**, 가동 보장이 없는 임시 주소입니다. 바뀌면 스킬 URL과 Test URL을 새 주소로 고치고 스킬을 다시 배포해야 합니다.
- 개발·시연 용도라면 지금 방식으로 충분합니다. 상시 운영이 목적이면 5단계의 클라우드 호스팅으로 옮겨야 합니다.
- 챗봇 응답 제한시간은 5초이므로, 무료 호스팅의 "잠자기 후 첫 요청 지연"이 큰 곳은 피해야 합니다.

## 1단계. 카카오톡 채널 만들기

챗봇은 카카오톡 채널에 연결해야 동작합니다. 이미 채널(law_consulting, @상속관련법)이 있으면 ①②은 건너뛰고 ③부터 확인하세요. 화면 문구는 카카오 업데이트에 따라 조금 달라질 수 있습니다.

1. 카카오 비즈니스에 카카오계정으로 로그인하고 비즈니스 회원가입을 합니다.
2. **새 비즈니스 만들기**를 선택하고 사업자 유형을 고릅니다. 사업자등록번호는 선택 사항이며, 없으면 일반 채널로 만들어지고 안내 문구가 표시됩니다.
3. 채널 정보를 입력합니다: 채널 이름, 검색용 아이디(예: `상속관련법` ), 카테고리.
4. 채널 관리자센터 **대시보드 > 기본 설정**에서 **[채널 공개하기]** 와 **[검색 허용하기]** 를 켭니다. 비공개 상태면 친구가 챗봇을 쓸 수 없습니다.
5. (선택) 메뉴 **메시지 > 웰컴 메시지**에서 채널 추가 시 보이는 인사말을 설정합니다. 챗봇 웰컴 블록과는 별개입니다.

> 채널이 2개면 좋습니다: **운영 채널**(실제 사용자용)과 **개발 채널**(테스트용). 두 채널은 같은 비즈앱에 연결되어 있어야 한 봇에 동시에 연결할 수 있습니다. 개발 채널이 후보에 안 뜨면 이 조건을 먼저 확인하세요.

## 2단계. Mac에서 스킬 서버와 터널 실행하기

터미널(zsh)에서 순서대로 실행합니다. 이미 설치된 항목은 건너뛰어도 됩니다.

### 2-1. 필수 도구 설치

```
# Homebrew 설치 여부 확인 (버전이 나오면 설치됨)
brew --version

# uv(파이썬 실행 도구) 설치
brew install uv

# cloudflared(터널) 설치
brew install cloudflared

# 설치 확인
uv --version
cloudflared --version
```

### 2-2. 프로젝트 폴더와 main.py 배치

```
mkdir -p ~/bot
cd ~/bot
# 앞서 보내드린 main.py를 이 폴더에 넣습니다
ls
```

### 2-3. 수동 실행 (터미널 창 2개 필요)

터미널 창 A — 스킬 서버:

```
cd ~/bot
uv run --with fastapi --with uvicorn uvicorn main:app --port 8001
```

터미널 창 B — 터널:

```
cloudflared tunnel --url http://localhost:8001
```

창 B 출력에서 `https://임의단어-임의단어.trycloudflare.com` 형태의 주소를 복사합니다. **스킬 URL = 그 주소 + `/skill`** 입니다. `/skill`을 빼면 405 오류가 납니다.

### 2-4. 한 번에 실행 (start.sh)

아래 스크립트를 `~/bot/start.sh`로 저장하면 위 두 창을 한 번에 실행하고 스킬 URL을 출력해 줍니다. (파일은 대화창에도 첨부했습니다.)

```
#!/bin/zsh
# 상속 길잡이 스킬 서버 + 터널 한 번에 실행 (main.py와 같은 폴더에 두세요)
cd "$(dirname "$0")" || exit 1
PORT=8001

lsof -ti tcp:$PORT | xargs kill 2>/dev/null
pkill -f "cloudflared tunnel" 2>/dev/null
sleep 1

trap 'kill $SERVER_PID $TUNNEL_PID 2>/dev/null; exit' INT TERM EXIT

uv run --with fastapi --with uvicorn uvicorn main:app --port $PORT >
server.log 2>&1 &
SERVER_PID=$!

for i in {1..30}; do
  curl -s http://127.0.0.1:$PORT/ >/dev/null && break
  sleep 1
done
curl -s http://127.0.0.1:$PORT/ >/dev/null || { echo "서버 시작 실패 → 
server.log 확인"; exit 1; }
echo "스킬 서버 OK (127.0.0.1:$PORT)"

cloudflared tunnel --url http://localhost:$PORT > tunnel.log 2>&1 &
TUNNEL_PID=$!

URL=""
for i in {1..30}; do
  URL=$(grep -o 'https://[a-z0-9-]*\.trycloudflare\.com' tunnel.log | head
-1)
  [ -n "$URL" ] && break
  sleep 1
done
[ -z "$URL" ] && { echo "터널 주소를 못 찾음 → tunnel.log 확인"; exit 1; }

echo ""
echo "================================================"
echo " 카카오 스킬 URL / 테스트 URL 에 넣을 주소:"
echo "   $URL/skill"
echo "================================================"
echo "(Ctrl+C 하면 서버와 터널이 모두 종료됩니다)"
wait
```

```
cd ~/bot
chmod +x start.sh
./start.sh
```

### 2-5. 동작 확인 (새 터미널 창에서)

```
# 로컬 서버 상태 ({"status":"ok"} 여야 함)
curl http://127.0.0.1:8001/

# 터널 경유 스킬 호출 — 응답시간과 200을 확인 (5초 이내여야 함)
curl -s -o /dev/null -w "%{time_total}초 / HTTP %{http_code}\n" -X POST
https://내터널주소.trycloudflare.com/skill \
  -H "Content-Type: application/json" \
  -d '{"userRequest":{"utterance":"안녕하세요"}}'
```

### 2-6. 종료·재시작

```
# 실행 중인 창에서 Ctrl+C  (start.sh 사용 시 둘 다 종료됨)

# 창을 이미 닫아 프로세스가 남아 있을 때 강제 정리
lsof -ti tcp:8001 | xargs kill
pkill -f "cloudflared tunnel"
```

재시작하면 터널 주소가 바뀝니다. 그때는 3단계의 3-3을 다시 하세요.

## 3단계. 챗봇 관리자센터에서 봇·스킬 만들고 연결하기

챗봇 관리자센터에서 진행합니다. 스킬 서버가 응답을 만들기 때문에, 블록은 **웰컴 블록과 폴백 블록 두 개만** 스킬에 연결하면 됩니다. 선택지마다 블록을 만들 필요가 없습니다.

### 3-1. 봇 만들기

1. 챗봇 관리자센터 접속 후 **봇 만들기 > 카카오톡 챗봇**을 선택하고 봇 이름(예: 상속 길잡이)을 입력합니다.
2. 만들어진 봇의 주소에 포함된 숫자열이 봇 ID입니다(예: `6acdaa44…`). 봇 ID는 주소에 들어 있을 뿐 스킬 서버 설정에는 따로 쓰지 않습니다.

### 3-2. 스킬 등록

1. 왼쪽 메뉴 **스킬 > 생성**을 누릅니다.
2. 입력값:

| 항목 | 입력값 |
| --- | --- |
| 스킬명 | 상속 길잡이 |

| 항목 | 입력값 |
| --- | --- |
| URL | `https://내터널주소.trycloudflare.com/skill` |
| Test URL | URL과 동일한 주소 (미배포 봇은 Test URL을 사용함) |

3. 저장 후 스킬 화면의 **스킬 테스트**에서 요청을 보내 응답 JSON(`version: 2.0`, `quickReplies`)이 나오는지 확인합니다. `405 METHOD_NOT_ALLOWED`가 나오면 URL 끝에 `/skill`이 빠진 것입니다.

### 3-3. 터널 주소가 바뀔 때(재시작 후)

1. **스킬** 목록에서 해당 스킬을 열어 URL과 Test URL을 새 주소 + `/skill`로 고칩니다.
2. 저장하고 스킬 테스트를 다시 통과시킵니다.
3. 운영 채널까지 반영하려면 스킬을 다시 배포하고 연결된 블록도 배포합니다(4단계).

### 3-4. 웰컴 블록 연결

1. **시나리오** 메뉴에서 **웰컴 블록**을 엽니다. 블록이 **사용중**으로 되어 있어야 합니다.
2. **파라미터 설정**(또는 스킬 선택 영역)에서 스킬 **상속 길잡이**를 선택합니다.
3. **봇 응답 > 응답 추가 > 스킬데이터**를 선택합니다. 기존 일반 텍스트 말풍선이 있으면 휴지통 아이콘으로 지웁니다(휴지통은 말풍선 왼쪽).
4. 저장합니다.

### 3-5. 폴백 블록 연결

웰컴 블록과 동일하게 폴백 블록에 스킬 **상속 길잡이** + **스킬데이터**를 연결합니다. 폴백은 키워드·선택지에 맞는 블록이 없을 때 모든 말을 받는 역할이므로, 이 두 개가 연결되면 사용자가 누르는 모든 바로연결 버튼(`messageText` 형태)과 직접 입력한 말이 전부 `main.py`로 전달됩니다.

### 3-6. 지식베이스(지식+)

지식+는 생성형 RAG가 아니라 FAQ 형태의 질문·답변 유사도 매칭이므로, 이 구성에서는 **미사용**으로 둡니다.

## 4단계. 봇테스트, 채널 연결, 배포

### 4-1. 봇테스트

1. 서버와 터널이 켜져 있는지 확인합니다(2-5의 curl).
2. 관리자센터 우측 상단 **봇테스트**를 열고 `안녕하세요`를 입력합니다. 웰컴 메시지와 바로연결 버튼(상속 절차 시작, 금융·채무, 상속세, 통신·구독, 위험 행동 확인, 법령·시행일 확인)이 나와야 합니다.
3. **발화 기록 목록의 항목이 아니라 말풍선 아래 노란 버튼**을 눌러 테스트합니다. 이전에 삭제한 블록의 발화 기록을 누르면 오류가 날 수 있습니다.
4. 서버 창에 `POST /skill ... 200`이 찍히면 연결이 정상입니다.

### 4-2. 운영·개발 채널 연결

두 채널은 역할이 다릅니다. 봇 ID는 채널 연결에 쓰이지 않고, 관리자센터에서 어느 봇인지 구분하는 주소에만 들어 있습니다.

| 구분 | 역할 | 배포 전 상태 |
|---|---|---|
| 운영 채널 | 실제 사용자에게 공개되는 채널 | 배포한 내용만 반영됨 |
| 개발 채널 | 테스트용 채널 | 저장한 내용을 바로 확인할 수 있음 |

1. **설정 > 챗봇 관리**에서 운영 채널과 개발 채널을 각각 연결합니다.
2. 개발 채널 후보가 나오지 않으면 해당 채널과 운영 채널이 **같은 비즈앱**에 연결되어 있는지 확인합니다 (카카오 공식 문서의 조건).
3. 개발 채널에서 먼저 카카오톡 대화로 테스트한 뒤 운영 채널에 배포하는 순서가 안전합니다.

### 4-3. 배포

1. 모든 변경사항(블록·스킬)을 **저장**합니다. 저장하지 않은 내용은 배포에 포함되지 않습니다.
2. 왼쪽 메뉴 **배포**에서 **전체 배포**를 실행합니다. 처음에는 전체 배포를 권장합니다.
3. 배포는 시작하면 중간에 멈출 수 없고 최대 30분 걸릴 수 있습니다.
4. **스킬 URL·Test URL을 바꾸었다면 스킬을 다시 배포하고, 그 스킬이 연결된 블록도 함께 배포**해야 운영 채널에 반영됩니다.
5. 삭제한 항목이 있으면 전체 배포가 필요합니다. 배포 이력은 배포 메뉴에서 확인합니다.

### 4-4. 카카오톡에서 최종 확인

1. 카카오톡에서 채널( `@상속관련법` )을 검색해 친구로 추가하고 1:1 채팅을 엽니다.
2. `안녕하세요` 를 보내고 바로연결 버튼을 하나씩 눌러 봅니다. 이 테스트도 로컬 서버와 터널이 켜져 있어야 합니다.

## 5단계. 컴퓨터를 꺼도 동작하게 하려면

세 가지 중 하나를 고릅니다.

| 방법 | 주소 고정 | 컴퓨터 켜둔 채로 | 비용 | 적합한 경우 |
|---|---|---|---|---|
| A. 현재(quick tunnel) | 아니오(재시작마다 변경) | 필요 | 무료 | 개발·시연 |
| B. 고정 터널(Named Tunnel) | 예(본인 도메인 필요) | 필요 | 터널 무료, 도메인 비용 | 주소 변경 없이 내 PC로 운영 |

| 방법 | 주소 고정 | 컴퓨터 켜둔 채로 | 비용 | 적합한 경우 |
|---|---|---|---|---|
| C. 클라우드 호스팅 | 예 | 불필요 | 서비스별 상이 | 상시 운영 |

### A. 컴퓨터가 잠들지 않게 유지 (현재 방식 보완)

```
# start.sh를 실행한 뒤, 새 터미널 창에서 절전 방지
caffeinate -s
```

### B. 고정 터널 (Cloudflare 계정 + 도메인 필요)

```
cloudflared tunnel login
cloudflared tunnel create sangsok-bot
cloudflared tunnel route dns sangsok-bot bot.내도메인.com
```

`~/.cloudflared/config.yml`:

```
tunnel: sangsok-bot
credentials-file: /Users/내계정명/.cloudflared/<터널ID>.json
ingress:
  - hostname: bot.내도메인.com
    service: http://localhost:8001
  - service: http_status:404
```

```
# 서버는 따로 실행한 뒤
cd ~/bot && uv run --with fastapi --with uvicorn uvicorn main:app --port 8001
# 터널 실행
cloudflared tunnel run sangsok-bot
```

스킬 URL은 `https://bot.내도메인.com/skill`로 한 번만 등록하면 됩니다.

### C. 클라우드 호스팅 (Render, Railway, Fly.io, Cloud Run 등)

필요한 파일과 시작 명령은 서비스와 상관없이 같습니다.

`requirements.txt`:

```
fastapi
uvicorn
```

시작 명령(Start Command):

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

배포 후 받은 HTTPS 주소 + `/skill`을 스킬 URL과 Test URL에 넣고 스킬을 다시 배포합니다. 무료 플랜은 일정 시간 요청이 없으면 서버가 절전되어 첫 요청이 5초를 넘길 수 있으므로, 시작 시간과 요금제는 각 서비스 문서에서 확인하세요. 또 호스팅 서비스는 외부에 데이터를 두는 것이므로 사용자 입력이 로그에 남는지 함께 점검하세요.

## 문제 해결과 매일 쓰는 명령

| 증상 | 원인 | 해결 |
|---|---|---|
| `zsh: command not found: cloudflared` | 미설치 | `brew install cloudflared` |
| `[Errno 48] address already in use` | 포트 사용 중 | `lsof -ti tcp:8001 \| xargs kill` 또는 `--port 8002`로 변경(터널 포트도 동일하게) |
| 스킬 테스트 405 METHOD_NOT_ALLOWED | URL 끝에 `/skill` 없음 | URL과 Test URL 모두 `/skill`까지 입력 |
| 봇테스트에서 오류 버튼 | 서버·터널 중단, 또는 삭제된 블록의 발화 기록 클릭 | 서버 창 확인 후 노란 바로연결 버튼으로 테스트 |
| 응답이 느리거나 실패 | 카카오 제한 시간 5초 초과 | `check.sh`로 응답시간 측정, 절전·네트워크 확인, 호스팅 시 콜드 스타트 확인 |
| 어제는 되던데 오늘 안 됨 | 터널 주소 변경 | 스킬 URL·Test URL 갱신 → 스킬 테스트 → 필요 시 스킬·블록 재배포 |
| 봇테스트는 되는데 카카오톡에서는 안 됨 | 미배포 또는 채널 미연결·비공개 | 4-2, 4-3 확인(저장 → 전체 배포, 채널 공개 설정) |

### 매일 작업 시작 순서

```
cd ~/bot
./start.sh                          # 서버 + 터널, 스킬 URL 출력
./check.sh https://출력된주소.trycloudflare.com   # 응답 확인
```

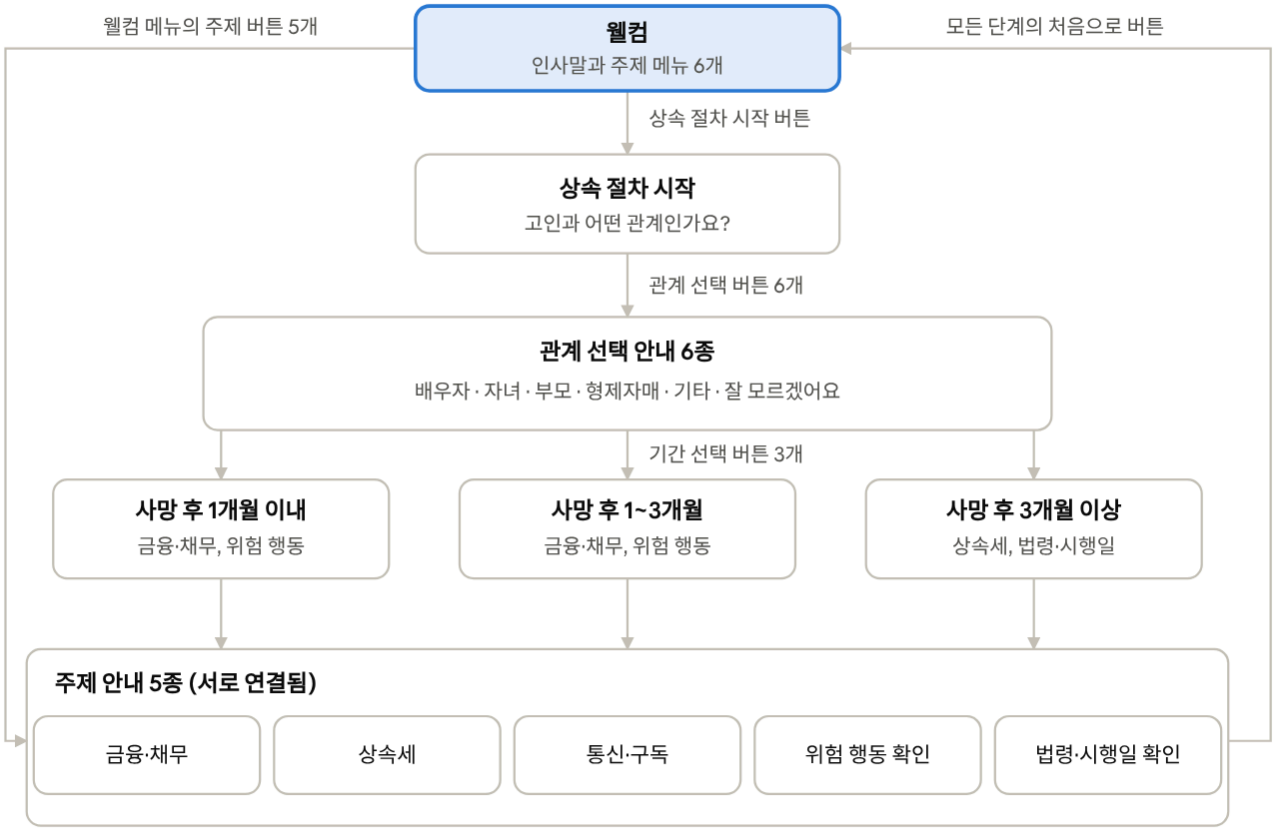
`check.sh`:

```
#!/bin/zsh
URL=${1:?"터널 주소를 인자로 주세요"}
curl -s -o /dev/null -w "응답시간 %{time_total}초 / HTTP %{http_code}\n" -X
POST "$URL/skill" \
  -H "Content-Type: application/json" \
  -d '{"userRequest":{"utterance":"안녕하세요"}}'
```

주소가 바뀌 때만 관리자센터의 스킬 URL을 고치고 재배포합니다.

## 채팅 흐름 구조도

**모든 대화는 웰컴에서 시작해 처음으로 버튼으로 돌아옵니다**




응답 16개: 웰컴 1 · 시작 1 · 관계 6 · 기간 3 · 주제 5 (모두 main.py 한 파일이 응답)

채팅 흐름 구조도 · 응답 16개, 5단계

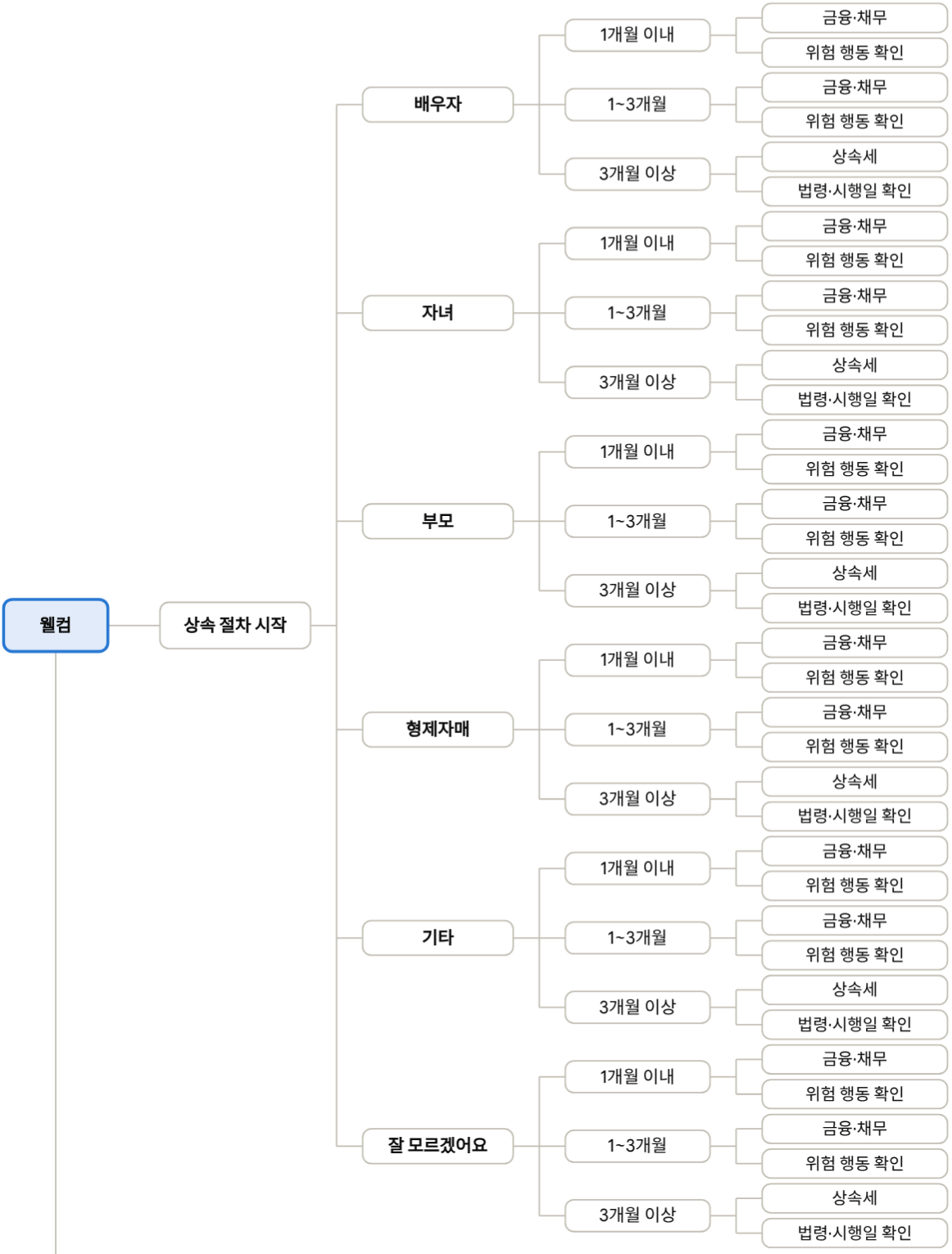
웰컴 메뉴에서 상속 절차 시작을 고르면 관계, 사망 후 기간, 주제 안내 순서로 이어집니다. 웰컴 메뉴의 주제 버튼 5개는 주제 안내로 바로 가고, 모든 안내의 처음으로 버튼은 웰컴으로 돌아갑니다.

# 채팅 경우의 수 트리

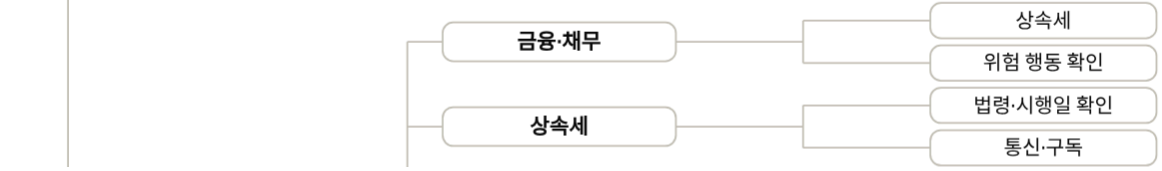
## 경우의 수를 모두 펼치면 상속 절차 시작에서 36가지, 주제 버튼에서 5가지로 갈립니다

웰컴 시작 관계 기간 주제 안내

### A. 상속 절차 시작 경로: 관계 6 × 기간 3 × 주제 2 = 36가지




### B. 웰컴 메뉴에서 주제로 바로 가는 5가지와 그 다음 버튼






모든 안내에 처음으로 버튼이 있어 웰컴으로 돌아갑니다.
A의 주제 안내 이후에도 B와 같은 다음 버튼이 이어집니다.

채팅 경우의 수 트리 · A 36가지, B 5가지

상속 절차 시작에서는 관계 6종, 기간 3종, 주제 2종이 곱해져 36가지로 갈리고, 관계에 따라 안내문은 다르지만 이후 버튼은 모두 같습니다. 웰컴 메뉴의 주제 버튼 5개는 관계와 기간을 건너뛰고 해당 주제로 바로 가며, 각 주제에서는 다음 버튼 2개가 이어집니다.

## 6단계. Render로 웹 배포 (컴퓨터 없이 상시 운영)

스킬 서버를 GitHub에 올리고 Render 무료 플랜에서 실행합니다. 이 프로젝트의 저장소는 `ksunnyoo/kakaogil`, 서비스 주소는 `https://kakaogil.onrender.com`입니다.

### 6-1. GitHub에 올리기 (Mac)

```
cd ~/bot
mv gitignore .gitignore        # 점 없는 이름으로 다운로드된 경우
git init
git config --global user.name "본인GitHub이름"
git config --global user.email "GitHub이메일"
git add main.py requirements.txt render.yaml .gitignore
git commit -m "상속 길잡이 스킬 서버"
git branch -M main
git remote add origin https://github.com/ksunnyoo/kakaogil.git
git push -u origin main
```

- `src refspec main does not match any` 오류는 커밋이 하나도 없다는 뜻입니다. `git add`에서 파일 이름(`.gitignore`)이 틀려 실패한 경우가 많습니다. `ls -a`로 파일을 확인하고 다시 커밋합니다.
- push가 인증에서 막히면 `brew install gh` 후 `gh auth login`으로 로그인합니다.

### 6-2. Render에서 배포

1. render.com에 GitHub 계정으로 가입하고 저장소 접근을 허용합니다.
2. **New + > Web Service**에서 `ksunnyoo/kakaogil`을 선택합니다.
3. 아래 값을 입력하고 배포합니다.

| 항목 | 값 |
| --- | --- |
| Branch | `main` |
| Runtime | Python 3 |
| Build Command | `pip install -r requirements.txt` |
| Start Command | `uvicorn main:app --host 0.0.0.0 --port $PORT` |
| Instance Type | Free |

4. 배포가 끝나면 주소가 표시됩니다. 예: `https://kakaogil.onrender.com`. `xxxx`는 예시 자리이므로 실제 주소를 써야 합니다.
5. 확인 (Mac):

```
curl https://kakaogil.onrender.com/
curl -s -X POST https://kakaogil.onrender.com/skill -H "Content-Type:
application/json" -d '{"userRequest":{"utterance":"안녕하세요"}}'
```

### 6-3. 카카오 스킬 URL 변경 및 배포

1. 챗봇 관리자센터 **스킬 > 상속 길잡이**의 URL과 Test URL을 모두 `https://kakaogil.onrender.com/skill`로 바꾸고 저장합니다.
2. **스킬 테스트**에서 `올바른 스킬 서버 응답입니다`가 나오는지 확인합니다.
3. 스킬을 배포하고, **배포 > 전체 배포**를 실행합니다. 배포 이력이 비어 있으면 카카오톡에서 응답하지 않습니다.
4. 코드를 고친 뒤에는 `git add . && git commit -m "수정" && git push`를 하면 Render가 자동으로 다시 배포합니다.

### 6-4. 채널 채팅 설정 (24시간 응답)

챗봇 관리자센터가 아니라 **카카오톡 채널 관리자센터**(business.kakao.com)에서 설정합니다.

1. 채널(@상속관련법)을 열고 왼쪽 메뉴 **1:1 채팅 > 채팅설정**으로 이동합니다.
2. 채팅 기능을 ON으로 합니다.
3. 채팅 가능 시간을 요일별로 00:00부터 23:59까지 설정합니다. 운영시간 밖에는 채팅불가능 안내 메시지가 나갑니다.

## 7단계. 절전 방지 (Render 무료 플랜)

Render 무료 서비스는 15분 동안 요청이 없으면 절전되고, 다시 깨어나는 데 약 1분(실측 33초)이 걸립니다. 카카오는 5초 안에 응답이 와야 하므로, 절전 중에 보낸 첫 대화는 실패합니다(`Request timeout ... after 5000 ms`). 주기적으로 서버 주소를 호출해 절전을 막습니다. Render 무료 인스턴스 시간은 월 750시간이므로, 서비스 하나는 24시간 켜 둘 수 있습니다.

| 방법 | 간격 | 비고 |
|---|---|---|
| cron-job.org (적용함) | 10분 | 무료. URL `https://kakaogil.onrender.com/`, GET |
| GitHub Actions (적용함) | 10분 | 보조용. 예약 실행이 밀릴 수 있음 |
| UptimeRobot | 5분 | 무료 플랜은 비상업적 용도만 허용 |

GitHub Actions 파일 `.github/workflows/keepalive.yml`:

```
name: keepalive
on:
  schedule:
    - cron: "*/10 * * * *"
  workflow_dispatch:
jobs:
  ping:
    runs-on: ubuntu-latest
    steps:
      - run: curl -fsS https://kakaogil.onrender.com/
```

```
cd ~/bot
mkdir -p .github/workflows
# 위 내용을 .github/workflows/keepalive.yml 로 저장한 뒤
git add .github/workflows/keepalive.yml
git commit -m "keepalive 워크플로 추가"
git push
```

절전이 막혔는지 확인하려면 요청을 아무것도 보내지 않고 20~30분 뒤 아래를 실행합니다. 1초 미만이면 정상입니다.

```
curl -s -o /dev/null -w "%{time_total}초 / HTTP %{http_code}\n"
https://kakaogil.onrender.com/
```

# 추가 문제 해결

| 증상 | 원인 | 해결 |
| --- | --- | --- |
| `Request timeout ... after 5000 ms` | Render 절전 상태에서 초기 기동 | `curl`로 서버를 깨운 뒤 재시도, 7단계 적용 |
| `src refspec main does not match any` | 커밋이 없음 | `git add` 파일명 확인(`.gitignore`) 후 커밋 |
| `Not Found` (curl) | 예시 주소(`xxxx`)를 그대로 사용 | Render가 부여한 실제 주소로 실행 |
| 봇테스트는 되는데 휴대폰이 안 됨 | 배포 이력 없음, 채팅 설정·운영시간 문제 | 6-3의 전체 배포, 6-4의 채팅 설정 확인 |
| 특정 문장(띄어쓰기·문장부호 포함)만 응답 안 됨 | 키워드 매칭 부족 | 개선된 `main.py`(문장부호 무시, 인사말 확장, `[skill]` 로그)로 교체 후 push |
| 서버 로그 확인 | 요청 도착 여부 확인 | Render **Logs**의 `[skill] block=... utterance=...` 줄 확인 |